IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 12 June 2024, accepted 5 September 2024, date of publication 16 September 2024, date of current version 27 September 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3461150

RESEARCH ARTICLE

MILE: Memory-Interactive Learning Engine for Neuro-Symbolic Solutions to Mathematical Problems

YUXUAN WU AND HIDEKI NAKAYAMA, (Member, IEEE)
Graduate School of Information Science and Technology, The University of Tokyo, Tokyo 113-8656, Japan

Corresponding author: Yuxuan Wu (wuyuxuan@nlab.ci.i.u-tokyo.ac.jp)

This work was supported by the Institute of AI and Beyond of the University of Tokyo, and JSPS KAKENHI Grant Number JP23K28139.

ABSTRACT Mathematical problem solving is a task that examines the capacity of machine learning systems to perform quantitative and logical reasoning. Existing work employed formulas as intermediate labels in this task to implement a neuro-symbolic approach and achieved remarkable performance. However, we are questioning the limitations of existing methods from two perspectives: the expressive capacity of formulas and the learning capacity of existing models. In this paper, we proposed the Memory-Interactive Learning Engine (MILE), a new framework for the neuro-symbolic solution to mathematical problems. The main contributions of this work include a new formula-representing technique and a new decoding method. In our experiment on the Math23K dataset, MILE outperformed existing methods on not only question-answering accuracy but also robustness and generalization capacity (the software is available at https://github.com/evan-ak/mile).

INDEX TERMS Mathematical reasoning, math word problem, neuro-symbolic method.

I. INTRODUCTION

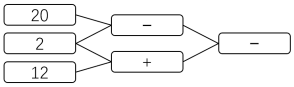
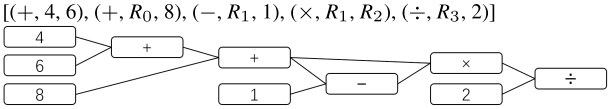
With the rapid development of deep learning in recent years, superhuman-level performance has been achieved in an increasing number of fields starting with computer vision and natural language processing. Not satisfied by these achievements, researchers have also begun to examine the capacity of machine learning models to perform abstract and logical reasoning. Mathematical reasoning is a field suitable for this study because of its high-level demand for analytical thinking. Early work in this field studied the performance of language models in performing end-to-end sequence-to-sequence prediction from question texts to answers, whereas their generalization capacity is not satisfying on relatively complicated problem categories represented by number theory and polynomials [1].

Recent work proposed the Chain-of-Thought (CoT) method to leverage the potential reasoning capability of Large Language Models (LLMs) by guiding the LLMs to generate step-by-step solutions to math problems with appropriate prompts [2], [3], [4]. However, due to the intrinsic mismatch between LLMs' probabilistic modeling in response generation and the demand for performing precise and symbolic calculations in math problems, CoT method is still troubled by the inconsistency in generated reasoning processes and LLMs' tendency to make arithmetic mistakes [5]. Another practice in mathematical reasoning is to implement a neuro-symbolic approach by making use of some intermediate labels known as formulas, templates, or equations [6], [7], [8], [9], [10]. Table 1 provides two examples of math problems and their corresponding formulas. Under this approach, machine learning models are only responsible for predicting formulas from questions, and these formulas can be calculated in a rule-based manner to acquire the final answers. These formulas endow machine learning models with notable capacities to formulate and organize abstract computing processes. Recent work also benefited from this practice to achieve remarkable performance in solving applied math problems.

However, in our study, we are also aware of some limitations exposed in existing work. The first limitation

The associate editor coordinating the review of this manuscript and approving it for publication was Binit Lukose.

TABLE 1. Two examples of mathematical problems together with their corresponding formulas and computing graphs. Note that the number and constant tokens in the formulas have been replaced with corresponding values for better readability. In the second example, prefix notation is not applicable because the computation is not tree-structured.

question	Andy has 12 apples. Bob has 20 apples. Bob gives 2 apples to Andy. How many more apples does Bob have than Andy now? **(Answer: 4)**
f_p	$[-, -, 20, 2, +, 12, 2]$
f_s	$[(-, 20, 2), (+, 12, 2), (-, R_0, R_1)]$
computational flow	
question	There are three classes in a school. Class one has 4 students. Class two has 6 students. Class three has 8 students. If every student shakes hands with each other in the school, how many times of handshakes happen in total? **(Answer: 153)**
f_p	N/A
f_s	$[(+, 4, 6), (+, R_0, 8), (-, R_1, 1), (\times, R_1, R_2), (\div, R_3, 2)]$
computational flow	

that we are concerned about is the expressive capacity of existing formula representations. Most existing work represents formulas with infix or prefix notations, whereas this practice serves under the assumption that the computation flows are tree-structured with algebraic operations at non-leaf nodes and raw numbers at leaf nodes. However, these tree-structured computations are inadequate for solving all math problems, and a simple exception is finding the greatest common divisor. The underlying reason for this defect is that tree-structured algebraic computation is not Turing-complete, and these representations do not truly support variables that can be referenced multiple times. This limits the expressive capacity of formulas. Moreover, another issue that raised our concerns in our study is that it seems that the existing formula prediction models may not truly understand the underlying concepts and natures of formulas. Their behavior is more similar to merely fitting the formulas encountered in the training stage. Some evidence for this proposition was obtained in our experiment, on which more discussion is going to be made in Section V.

Considering these limitations of existing work, in this study, we are motivated to put forward a new paradigm for formula representation and prediction in neuro-symbolic solutions to math problems. We first introduce a new practice for formula representation named segmented representation, and then propose the Memory-Interactive Learning Engine (MILE), a new framework for formula learning that is suitable for our segmented representation. The major contributions of this work can be summarized as follows:

- We introduced a new formula-representing technique that supports non-tree-structured formulas.
- We proposed a new decoding method that is compatible with this new formula representation and outperforms the formula prediction models developed previously.
- We illustrated that our proposed method exhibits enhanced robustness and generalization capacity than existing approaches.

II. RELATED WORK

A. MATHEMATICAL REASONING TASKS

Mathematical reasoning is a field that examines the capacity of machine learning models to perform abstract, quantitative, and logical reasoning on mathematical problems. In this field, the solution to math problems described in natural languages is most widely studied in recent years, and many relevant datasets have been published. Math23K [6] is a dataset crawled from several websites consisting of 23,162 applied math problems with formula annotations. MathQA [11] is a dataset consisting of 37,200 math problems collected from another previously developed dataset AQuA [12]. However, the formula annotations for MathQA are imperfect and contain noise [13], [14], which affects the effectiveness of formula learning. Mathematics [1] is a comprehensive dataset providing problems generated in various mathematical areas. Nevertheless, this dataset is developed for studying the end-to-end reasoning capacity of language models, so no formula annotations are involved. GSM8K [2] is a dataset that contains 8,792 math problems together with their solutions in natural languages. This dataset has been used to study the chain-of-thought approach to solving math problems in recent work [3], [4].

B. MATHEMATICAL PROBLEM SOLVERS

Besides the end-to-end methods that are examined early on, modern approaches to solving math problems generally fall into two main categories: formula-based methods and Chain-of-Thought (CoT) methods. Among them, formula-based methods implement a neuro-symbolic approach by employing formulas to formulate the computing and reasoning processes. This approach was first proposed by Wang et al. [6] and then improved in later work [7], [8], [9], [10]. Formula-based methods have achieved remarkable performance on challenging mathematical reasoning datasets represented by Math23K. Recent work also made efforts to take advantage of LLMs and proposed sampling techniques to further improve the performance [15], [16].

Jie et al. introduced a new approach to deductive expression generations [17].

On the other hand, CoT is a novel practice that manages to solve math problems step-by-step. This method acquires solutions to problems by generating each intermediate reasoning step with LLMs continuously, and they have been successfully implemented in recent work by fine-tuning LLMs or performing few-shot prompting [2], [3], [4]. Some work has also shown that LLMs pretrained on a huge corpus can even be zero-shot reasoners [18], [19], [20]. These methods can also be integrated with external tools such as programmed calculators or scientific computing APIs [2], [21]. However, in consideration of the benefits of formula-based methods including lower computational costs, better controllability, and their compatibility with arbitrary sets of operations without fine-tuning the whole LLMs, we still consider formula-based methods quite valuable as a neuro-symbolic approach in the mathematical reasoning task and regard them as the main focus of this study.

C. NEURAL NETWORKS WITH EXTERNAL MEMORY

Neural Networks with external memory, or what is known as the Neural Turing Machine, is an approach that endows Recurrent Neural Networks (RNNs) with the capability to write to or read from external memory [22], [23]. These approaches are proposed with the purpose of enhancing the capacity of models to maintain long-term information. The basic architecture of these models can be described as a recurrent controller network along with an external memory space, which was the basis of our idea of introducing memory mechanisms to MILE.

III. FORMULA REPRESENTATION

In formula-based methods, given a question denoted by q, the machine learning model is expected to predict a formula f for q. The formula f can be formulated in multiple different manners, while infix and prefix notations are most commonly adopted in existing work [6], [7]. In the following discussion, we take prefix notations as examples, whereas infix notations just work under a similar principle. We let f_p denote the formula f represented in the prefix notation to distinguish it from our representations. Generally, f_p is made up of a sequence of tokens $[t_0,\ldots, t_n]$. Among them, each token t_i is an element drawn from the union of three sets: T_{op}, T_{num}, and T_{con}. Here, T_{op} is the set of operator tokens, of which the elements are relevant to specific tasks and datasets. A simple and typical example of T_{op} is the set of the four basic arithmetic operations $\{+, -, \times, \div\}$. T_{num} is the set of number tokens $\{N_0,\ldots, N_m\}$ that establish references to the numbers appearing in the question text. During data preprocessing, the numbers in the question text are extracted as $[n_0,\ldots, n_m]$. In the calculation of formulas, N_i is replaced by the i-th extracted number n_i. T_{con} is the set of constant tokens that refer to the constants that are crucial to solving some problems but may not explicitly appear in the question text. The composition of T_{con} is also relevant to specific tasks and datasets. An example can be $\{1, \pi, 60, 100\}$.

Nevertheless, as we indicated above, this prefix notation f_p can only be utilized to represent tree-structured computations, which limits the expressive capacity of formulas. To address this problem, on the basis of the formula annotations adopted by Amini et al. [11], we propose the segmented representation f_s, a new practice for formula representation that is not restricted to tree-structured computations.

Generally, the basic form of this representation is a sequence of terms $[S_0,\ldots, S_n]$. Among them, each term S_i contains one operator and a certain number of operands. That is, with *opr* denoting the operator and *opd* denoting the operands, $S_i = (opr, opd_1,\ldots, opd_k)$. Here, the number of operands k is specific to the operator. For example, arithmetic operations take two operands and trigonometric operations take one. Compared with the prefix notation, the operator *opr* in f_s is generated from the same operator set T_{op}, whereas the operand *opd* is generated from the union of T_{num}, T_{con}, and another particular set T_{ref}. Here, $T_{ref} = \{R_0,\ldots, R_{n-1}\}$ is the set of reference tokens that establish references to the intermediate computing results of previous terms. By the time of calculation, each term of f_s is calculated from S_0 to S_n in order, and the last term S_n provides the final result of the whole formula. These formulas can also be considered as an instance of functional Domain Specific Languages (DSL) with *opr* as the function name and $(opd_1,\ldots, opr_k)$ as the function arguments. Compared with the formula representations with infix and prefix notations employed in existing work, the most noteworthy characteristic of our segmented representation lies in its compatibility with non-tree-structured computations.

IV. MEMORY-INTERACTIVE LEARNING ENGINE

Generally, MILE follows the encoder–decoder architecture as shown in Figure 1. Its computation of MILE can be formulated with Equations 1 and 2.

$$e_e, e_s = Encoder(q) \tag{1}$$

$$f = Decoder(e_e, e_s) \tag{2}$$

MILE is not selective to the encoder, as long as it encodes the question q to two tensors: the token embedding e_e and the sentence summary e_s. Here, $e_e \in \mathbb{R}^{L\times d_e}$, where L denotes the input sequence length and d_e denotes the embedding dimension. $e_s \in \mathbb{R}^{d_s}$, where d_s denotes the summary dimension. Owing to its compatibility, MILE can work on any common encoders such as LSTM [24], BERT [25], or GNN [10], [26]. As for the decoding, MILE implements a recurrent decoding process while interacting with a memory space named Memory Embedding Pool.

A. MEMORY EMBEDDING POOL

As its name indicates, the most distinctive design of MILE is a latent space of memory that we call the Memory Embedding

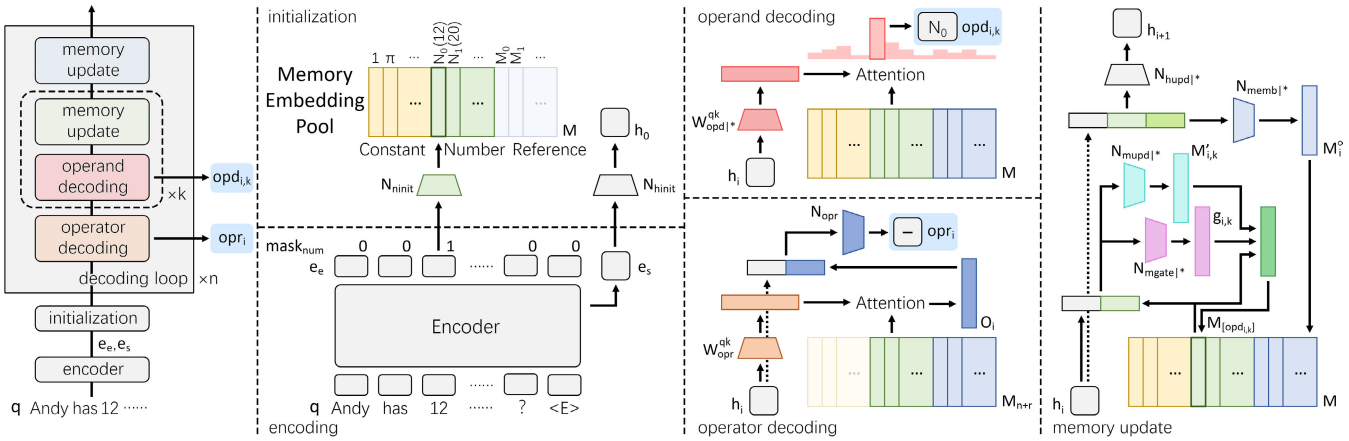


FIGURE 1. The overview of our Memory-Interactive Learning Engine.

Pool. This design is based on existing work that extended RNNs with external memory [23] and implemented the Neural Turing Machine [22]. Following a similar principle, our Memory Embedding Pool is developed to assist the model in better maintaining long-term information.

Generally, our Memory Embedding Pool is composed of three types of different embeddings corresponding to the three types of available operands: constant embedding, number embedding, and reference embedding. It established a unified embedding space for all these available operands, of which the underlying insight is that these operands should share equal status and be treated equally throughout the decoding even though their origins are different. With M denoting the Memory Embedding Pool, $M \in \mathbb{R}^{[W_c+W_n+W_r]\times d_m}$. Here, d_m denotes the memory embedding dimension. W_c, W_n, and W_r are the lengths of T_{con}, T_{num}, and T_{ref}, respectively. They are also referred to as the widths of memory embeddings. We generated the three types of embedding under the following principles.

1) CONSTANT EMBEDDING

We simply treat constant embedding as trainable embedding vectors shared across all questions. That is, with M_c denotes the constant embedding, $M_c \in \mathbb{R}^{W_c \times d_m}$ is directly a parameter to be optimized in our model.

2) NUMBER EMBEDDING

We generate the number embedding specific to each question on the basis of its encoder embedding e_e. They are calculated after the encoding is finished and before the decoding is started. Details are presented in *Initialization* in Section IV-B.

3) REFERENCE EMBEDDING

We generate the reference embedding continuously after the decoding of each formula term. That is, their width W_r starts with zero and increases as the decoding proceeds. Details are presented in *Memory Update* in Section IV-B.

B. FORMULA DECODING

As shown in Figure 1, after initialization, the decoding of a formula is a recurrent process with each formula term $S_i = (opr_i, opd_{i,1}, \ldots, opd_{i,k})$ as the basic decoding unit. In this process, each decoding loop contains three subprocedures: operator decoding, operand decoding, and memory update. Their working principles are presented as follows.

1) INITIALIZATION

We first initialize the hidden state $h \in \mathbb{R}^{d_h}$ and number embedding $M_n \in \mathbb{R}^{W_n \times d_m}$ at the beginning of decoding. They are obtained through Equations 3 to 6.

$$h_0 = N_{hinit}(e_s) \tag{3}$$

$$mask_{num} = [1 \; if \; \text{is\_num}(t_i) \; else \; 0 \; for \; t_i \; in \; q] \tag{4}$$

$$e_{e|num} = e_e.\text{gather}(mask_{num}) \tag{5}$$

$$M_n = N_{ninit}(e_{e|num}) \tag{6}$$

Here, N_{hinit} and N_{ninit} are multilayer perceptrons (MLPs). is_num() judges whether a token t_i in q is a number token. gather() gathers the embedding vectors on number tokens along the axis of sequence length so that $e_{e|num} \in \mathbb{R}^{W_n \times d_e}$.

2) OPERATOR DECODING

The decoding process of operators is described by Equations 7 to 9. Here, M_{n+r} is the variable part of the memory embedding, which contains the number embedding and reference embedding and starts from the end of the constant embedding. $O_i \in \mathbb{R}^{d_m}$ is an observation across M_{n+r} and is obtained simply with the attention mechanism, where $W^{qk}_{opr} \in \mathbb{R}^{d_h \times d_m}$ denotes the product of query and key matrices. Finally, O_i is concatenated to h_i to be fed to the MLP classifier N_{opr} to predict the operator opr_i.

$$M_{n+r} = M_{[W_c:]} \tag{7}$$

$$\begin{aligned} O_i &= \text{Attention}(h_i, M_{n+r}) \\ &= softmax(h_i W^{qk}_{opr} M^\top_{n+r}) M_{n+r} \end{aligned} \tag{8}$$

$$opr_i = \text{argmax}_{T_{op}} N_{opr}([h_i, O_i]) \tag{9}$$

3) OPERAND DECODING

The decoding process of operands is described by Equations 10 and 11. Note that this process is repeated k times in each decoding loop because each operator is associated with k operands, while for most arithmetic operations, $k = 2$.

$$P_{i,k} = softmax(h_i W^{qk}_{opd|opr_i,k} M^\top) \tag{10}$$

$$opd_{i,k} = \text{argmax}_{T_{con} \cup T_{num} \cup T_{ref}} P_{i,k} \tag{11}$$

Here, $P_{i,k}$ is the likelihood on each memory embedding terms of being the desirable k-th operand for operator opr_i. Again, $W^{qk}_{opd|opr_i,k} \in \mathbb{R}^{d_h \times d_m}$ denote the product of query and key matrices. However, unlike W^{qk}_{opr}, which is a single parameter shared throughout the entire decoding process, there is a set of $W^{qk}_{opd|*} = \{W^{qk}_{opd|opr_0,1}, \ldots, W^{qk}_{opd|opr_n,k}\}$ corresponding to each operand position of each available operator in T_{op}. An exception here is that for commutative operations such as addition and multiplication, a single $W^{qk}_{opd|*}$ is shared by the symmetric operands so that $W^{qk}_{opd|+,1} \equiv W^{qk}_{opd|+,2}$. This design helps the model to better learn the underlying concepts of mathematical operations and prevent overfitting.

4) MEMORY UPDATE

After the operator and operands in each formula term S_i are decoded, we update the Memory Embedding Pool by two means. First, we update the embedding of the memory terms that are inferred as operands. Then, we generate an embedding for the current formula term S_i and append it to the reference embedding. Note that the first update happens every time after each operand is decoded, whereas the second update happens only once at the end of the current decoding loop. The update of the memory terms inferred as operands follows Equations 12 to 14.

$$M'_{i,k} = N_{mupd|opr_i,k}([h_i, M_{[opd_{i,k}]}]) \tag{12}$$

$$g_{i,k} = N_{gate|opr_i,k}([h_i, M_{[opd_{i,k}]}]) \tag{13}$$

$$M_{[opd_{i,k}]} = g_{i,k} M'_{i,k} + (1 - g_{i,k}) M_{[opd_{i,k}]} \tag{14}$$

Here, M is updated solely on its $opd_{i,k}$ slice $M_{[opd_{i,k}]}$. Its update follows a gate mechanism similar to that in GRU [27]. $N_{mupd|opr_i,k}$ and $N_{gate|opr_i,k}$ are the MLPs that calculate the update and gate vectors, respectively, with the original $M_{[opd_{i,k}]}$ concatenated to h_i as inputs. Similarly to $W^{qk}_{opd|*}$, there are also two sets of $N_{mupd|*}$ and $N_{mgate|*}$ corresponding to each operand position of each operator, and the ones for symmetric operands are shared.

The generation of the new memory embedding for S_i follows Equations 15 and 16.

$$M^\circ_i = N_{memb|opr_i}([h_i, M_{[opr_{i,1}]}, \ldots, M_{[opr_{i,k}]}]) \tag{15}$$

$$M = [M, M^\circ_i] \tag{16}$$

Here, $N_{memb|opr_i}$ is the MLP arising from set $N_{emb|*}$ that calculates the new embedding vector M°_i. It takes as input the concatenation of h_i and the k associated original memory embedding $M_{[opd_{i,1}]}$ to $M_{[opd_{i,k}]}$. M°_i is concatenated to M then, which makes the width of reference embedding W_r increase by one in each decoding loop.

Finally, the hidden state is updated through Equation 17.

$$h_{i+1} = N_{hupd|opr_i}([h_i, M_{[opr_{i,1}]}, \ldots, M_{[opr_{i,k}]}]) \tag{17}$$

In summary, all the trainable parameters and modules in the decoding side of MILE include M_c, N_{hinit}, N_{ninit}, W^{qk}_{opr}, N_{opr}, $W^{qk}_{opd|*}$, $N_{mupd|*}$, $N_{mgate|*}$, $N_{memb|*}$, and $N_{hupd|*}$.

C. FORMULA MUTATION

Considering that the decoding mechanism of MILE is relatively more complex than those of existing approaches such as the RNN decoder and the Tree decoder [10] and MILE has more parameters, the training of MILE is more challenging. Moreover, due to the cost of crafting formula annotations, most publicly available mathematical reasoning datasets with formula annotations only contain from several thousand to tens of thousands of training samples. In our experiment, we also found that such a limited amount of data may not satisfy the training demand for MILE. As a result, we propose formula mutation, a data augmentation technique to expand the scale of training data.

The idea of formula mutation is straightforward: generating functionally equivalent formulas. This can also be easily implemented on formulas in segmented representation, for which we simply swap the symmetric operands in each formula term performing commutative operations. More formally, with $f_s = [S_0, \ldots, S_n]$, $S_i = (opr_i, opd_{i,1}, \ldots, opd_{i,k})$, $T_{op|c}$ denoting the set of commutative operators, and $\prod$ performing the Cartesian product, the mutation on f_s is given by Equations 18 and 19. Table 2 provides an example of the mutation results on a formula with commutative operations.

$$S'_i = \begin{cases} \{S_i\}, & \text{if } opr_i \notin T_{op|c} \\ \{(opr_i, opd_{i,1}, opd_{i,2}), (opr_i, opd_{i,2}, opd_{i,1})\}, & \text{if } opr_i \in T_{op|c} \end{cases} \tag{18}$$

$$\text{Mutation}(f_s) = \prod_{i=0}^{n} S'_i \tag{19}$$

Formula mutation is utilized by default in MILE, and it can also be applied to other decoding methods by transforming the formula in the segmented representation f_s back to the prefix notation f_p. We conducted contrast experiments to investigate the effect of formula mutation and report our interesting findings in Section V-B.

V. EXPERIMENTS

In our experiments, we studied the performance of MILE and compared MILE with existing decoding methods on the Math23K dataset [6]. We selected Math23K as the dataset to conduct our experiment on because it is the largest officially published mathematical reasoning dataset with complete and

TABLE 2. Example of the mutation results on a formula.

original formula	$[(+, N_0, N_1), (-, N_2, R_0), (\times, R_1, N_3)]$
mutation results	$[(+, N_0, N_1), (-, N_2, R_0), (\times, R_1, N_3)]$ $[(+, N_1, N_0), (-, N_2, R_0), (\times, R_1, N_3)]$ $[(+, N_0, N_1), (-, N_2, R_0), (\times, N_3, R_1)]$ $[(+, N_1, N_0), (-, N_2, R_0), (\times, N_3, R_1)]$

clean formula annotations. Math23K is also the dataset on which most existing work reported their results, which makes it a convenient benchmark for comparison. Appendix A provides more explanations for our considerations in dataset selection. We show the experimental setup and primary results in Sections V-A and V-B, and present our findings in further studies in Sections V-C and V-D.

A. EXPERIMENTAL SETUP

As we indicated above, MILE has an encoder–decoder architecture. For the encoder, we compared the LSTM encoder [24], the graph encoder [10], BERT [25], and RoBERTa [28]. For the decoder, we compared our method with the LSTM decoder and the tree decoder [10]. Note that MILE predicts formulas in segmented representation f_s, whereas other decoders predict formulas in prefix notation f_p. The implementation details are presented in Appendix B. We evaluate the performance of models in terms of the accuracy of final answers acquired from predicted formulas. Note that this accuracy is different from and is not directly comparable with the formula matching accuracy applied in some existing work, which does not tolerate the functionally equivalent formulas that lead to the same answers.

B. PRIMARY RESULTS

Table 3 shows the question answering accuracy of baselines and MILE on the Math23K dataset under 5-fold cross-validation and on a public test set. Here, DNS is the original RNN-based learning model proposed by Wang et al. [6]. T-RNN [7] predicts equation templates in tree structure with recursive neural networks. TreeDecoder [8] implements a tree-structured decoder to predict prefix templates. GTS [9] employs tree-structured neural networks to predict expression trees in a goal-driven manner. Graph2Tree [10] implements a graph-based encoder and a tree-based decoder to further improve the performance of prediction. DeductReasoner [17] introduces a novel mechanism to generate expressions deductively. What follows are the results of MILE on four different encoders. Some other work represented by [15] is not included in the comparison here because their contributions are not the prediction model itself and thus do not conflict with our proposal.

As for these results, it should first be noted that because BERT and RoBERTa included external corpus data in their training stage, the accuracies achieved by models using BERT or RoBERTa as their encoders are not directly comparable with the others. Among the implementations without pretrained encoders, MILE on the graph encoder achieved basically the same performance as Graph2Tree. When the BERT encoder is employed, the performance of MILE is comparable to the DeductReasoner. On the RoBERTa encoder, MILE achieved the best result and outperformed all the existing baselines. We explain these results on the basis of the design of MILE. Given that the decoding mechanism of MILE is relatively more complex, and MILE also contains components, such as $W^{qk}_{opd|*}$, that are specific to every single valid operator, these characteristics make the training of MILE more difficult and can get overfitted more easily. Moreover, these challenges can be further magnified by the insufficiency of training data, which is the very situation of Math23K, where there are only about 18k training samples under 5-fold cross-validation. However, these problems can be largely mitigated by BERT and RoBERTa, which are pretrained on a large enough corpus. A well-pretrained encoder can make MILE focus on the training of the decoding side and thus achieve better performance.

On the other hand, we studied the impact of the formula mutation technique introduced in Section IV-C. The results are shown in Table 4. From these results, interestingly we found that MILE is the only model that consistently benefits from the formula mutation. From the perspective of human beings, the formula mutation is quite a natural practice, and the functionally equivalent formulas would not adversely affect our learning of arithmetic. Conversely, the variety of formulas imparts to us a better understanding of the nature of mathematics. However, this property does not hold for the existing formula predicting models, which further indicates that these models may not truly grasp the underlying concepts and natures of formulas and are simply fitting the formulas in the training set. In contrast, the capability of MILE to learn from mutated formulas suggests that MILE learns and predicts formulas in a way more natural and close to human beings, which explains the reason why MILE outperforms other decoding techniques from another aspect. To provide more evidence for this proposition, we conducted further studies and report the results in the following sections.

C. GENERALIZATION CAPACITY

The first further question we want to investigate is the generalization capacity of different formula prediction models. For this goal, we conducted experiments on different train–test set splits. For the primary experiment presented in Section V-B under 5-fold cross-validation and on the public test set, the split of the training and test sets was conducted randomly. Whereas, in this experiment, we split these two sets with two different metrics: *number count* and *formula length*. For the *number count* split, we sort the (q, f) pairs by the count of number tokens in q. Then with the proportion p, we use p of samples with more number tokens in q as test data, and the rest (1-p) of samples with fewer number tokens in q as training data. This split is denoted by *number-p* for short. For

TABLE 3. Question answering accuracy of baselines and MILE on the Math23K dataset under 5-fold cross-validation and on a public test set.

	5-fold	public
DNS [6]	58.1%	-
T-RNN [7]	66.9%	-
TreeDecoder [8]	-	69.0%
GTS [9]	74.3%	75.6%
Graph2Tree [10]	75.5%	77.4%
DeductReasoner / BERT † [17]	82.6%	84.5%
DeductReasoner / RoBERTa † [17]	83.0%	85.1%
MILE / LSTM	65.3%	66.1%
MILE / Graph	75.4%	77.5%
MILE / BERT †	83.1%	84.1%
MILE / RoBERTa †	**84.5%**	**85.2%**

† implementations that employ pretrained encoders.

TABLE 4. Question answering accuracy of different model implementations [without / with] formula mutation under 5-fold cross-validation.

Enc. \ Dec.	LSTM	Tree	MILE
LSTM	65.2% / **65.5%**	**68.9%** / 68.0%	63.1% / **65.3%**
Graph	**71.2%** / 69.6%	**75.5%** / 72.0%	73.5% / **75.4%**
BERT	**78.0%** / 75.9%	**81.7%** / 81.3%	81.6% / **83.1%**
RoBERTa	**79.9%** / 77.6%	**83.0%** / 82.4%	83.2% / **84.5%**

TABLE 5. Question answering accuracy of different inference models under different train–test set splits.

split	BERT / LSTM	BERT / Tree	MILE / BERT
5-fold	78.0%	81.7%	**83.1%**
number-0.2	13.4%	23.5%	**24.9%**
number-0.3	31.1%	39.6%	**41.5%**
number-0.4	39.3%	46.0%	**47.7%**
formula-0.2	21.4%	24.0%	**25.4%**
formula-0.3	4.0%	3.8%	**4.1%**
formula-0.4	17.9%	19.2%	**19.8%**

the *formula length* split, we sort the (q, f) pairs by the length of f (i.e., the count of terms S_i in f). Then with the proportion p, we use p of samples with longer f as test data, and the rest (1-p) of samples with shorter f as training data. This split is denoted by *formula*-p for short. All the other experimental settings except for the train–test set split are the same as those in the primary experiment. The results are shown in Table 5.

From these results, it can first be noted that there is a huge gap between the accuracies achieved in 5-fold cross-validation and our experimental splits. This is because our splits produce a highly challenging test environment, where the models have to generalize to problems that have more numbers involved and require longer calculating flows to solve than all the problems seen in the training stage. However, even in this challenging environment, MILE shows the best performance among the methods compared, which verified its superior generalization capacity. More detailed analyses of the results and a case study are presented in Appendix C,

TABLE 6. Question answering accuracy of different inference models with token replacement. We are unable to conduct this experiment on the tree decoder, and the reason is explained in Appendix B.

	BERT / LSTM	MILE / BERT
original	78.0%	**83.1%**
replace-2x	30.6%	**74.1%**
replace-3x	10.3%	**70.9%**

D. ROBUSTNESS

The robustness of different formula predicting models is another question that raised our concerns. To shed light on this question, we designed an experiment in which the word tokens in question texts can be randomly replaced by number tokens. That is, with $q = [t_0, \ldots, t_n]$, we have $\{t^{\neg num}\} = \{t_i \; for \; t_i \; in \; q \; if \; not \; \text{is\_num}(t_i)\}$ denoting the set of non-number tokens. For each q, we randomly sample r tokens from $\{t^{\neg num}\}$ and replace them with r random tokens sampled from $\{t^{num}\}$. Here, $\{t^{num}\}$ denotes the collection of all number tokens from all q in the dataset. That is, $\{t^{num}\} = \cup_{q_i}\{t_j \; for \; t_j \; in \; q_i \; if \; \text{is\_num}(t_j)\}$. In the experiment, we decided r for each q individually in line with m, the count of number tokens in q. As for the two experimental settings *replace*-2x and *replace*-3x, we let $r = 2m$ and $r = 3m$, respectively. An additional restriction here is that at least half of the word tokens in each q will be kept unchanged. After this replacement, the formulas are also updated correspondingly to keep the references to numbers accurate. The other settings in this experiment are the same as those in the primary experiment. The results are shown in Table 6.

From these results, it is shown that the performance of the LSTM decoder is affected by the token replacement significantly. The reason here is that the number tokens T_{num} distinguish numbers appearing in question texts by their indexes in a hard-coded way, which is highly sensitive to the absolute indexes of numbers rather than their relative contexts. As a result, this coding fails when some word tokens are replaced by number tokens in the question texts, which changes the absolute indexes of number tokens. This problem can be mitigated to some extent by performing a cross-attention mechanism with encoders [9], [10]. Nevertheless, we go one step further in the design of MILE, where the decoding of operands is fully driven by the attention calculation upon the Memory Embedding Pool. Owing to this feature, the impact of token replacement is notably reduced on MILE.

VI. CONCLUSION

In this paper, we discussed the limitations of existing neuro-symbolic approaches to solving mathematical problems from two aspects: the expressive capacity of formulas and the learning capacity of models. To address these problems, we first introduced the segmented representation technique to support non-tree-structured formulas, and then proposed MILE to implement reasonable formula predictions

with the segmented representation. Our experiment on the Math23K dataset verified the effectiveness of our proposed method and illustrated its superiority in terms of generalization capacity and robustness. The potential limitations of MILE may lie in the difficulty of its training. In view of this, we proposed the formula mutation for data augmentation and suggested employing pretrained Language Models as the encoder. These practices are also feasible in mitigating the training difficulty in similar tasks and datasets. On this basis, we consider our proposals a valid and advanced approach to neuro-symbolic mathematical problem solving. We also hope that these methods can be applied to more general neuro-symbolic approaches adopting functional DSL as symbolic labels.

APPENDIX A
DATASET SELECTION

As we discussed in Section II-A, many datasets have been published in recent years for studying the capacity of machine learning models to perform mathematical reasoning tasks from different aspects. However, we consider that not all of them are appropriate for the study and experiments in this work. Table 7 provides a comparison of several common datasets for mathematical reasoning tasks. Compared to Math23K [6], early datasets represented by MAWPS [29] contain relatively too few samples for training large learning models nowadays. Later datasets represented by MathQA [11] and GSM8K [2] include more training samples. However, the noise in their annotations also results in the insufficiency of valid training samples ultimately after data cleaning. Table 8 provides some examples of the invalid formula annotations and solutions in these datasets. Mathematics [1] provides high-quality and theoretically infinite training samples as a generative dataset. Nevertheless, formula annotations are not adopted in this dataset, which makes this dataset inapplicable for studying formula-based methods. On this basis, we consider Math23K the largest mathematical reasoning dataset with complete and clean formula annotations that are publicly available for now, and the dataset most suitable for our study and experiments.

TABLE 7. The comparison of several common datasets for mathematical reasoning task in terms of the availability of formula annotations and training samples.

	total training samples	formula annotations	valid training samples
MAWPS [29]	3,320	clean	3,320
Math23K [6]	23,162	clean	23,162
MathQA [11]	29,660	noisy	11,930
Mathematics [1]	generative	N/A	
GSM8K [2]	7,473	noisy	5,745

APPENDIX B
IMPLEMENTATION DETAILS

In our experiments, for the LSTM encoder, we employed a two-layer Bidirectional LSTM [30] with hidden state size 256. For the LSTM decoder, we employed a two-layer LSTM with hidden state size 512. For the graph encoder and the tree decoder, we followed the default implementations and settings in the published code of [10]. For BERT, we used the pretrained Chinese BERT [31] because all the questions in Math23K are written in Chinese. For RoBERTa, we used the model provided by Cui et al. [32], [33]. For MILE, we have the memory embedding dimension $d_m = 128$ and the dimension of hidden state $d_h = 512$. We employed two-layer MLPs for N_{hinit}, N_{ninit}, N_{opr}, $N_{mupd|*}$, and $N_{memb|*}$ with hidden layer dimension the same as output layers. We employed single layer fully connected networks for $N_{mgate|*}$ and $N_{hupd|*}$. The dimension of the query and key vectors of W^{qk}_{opr} and $W^{qk}_{opd|*}$ is 128. We optimized all the models using the Adam optimizer with its recommended parameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$ [34]. The learning rate is 10^{-4} for all the parameters in MILE's decoding side, and 10^{-3} for parameters in all other parts of models. The models are trained with batch size 128 over 160 epochs. Learning rate decay is conducted twice at epochs 80 and 120 with a multiplier of 0.2.

As for the decoding, we let $T_{op} = \{+, -, \times, \div, =\}$ and $T_{con} = \{1, 2, \pi, 100\}$ in line with the demand of Math23K. Note that the operator $=$ in T_{op} is a special token included to support the case that the answers to some questions can directly be a number appearing in the question texts. There is no arithmetic calculation required in these cases, so we formulate f_s with $[(=, N_i)]$. We employed beam search with beam size 5 for the inference.

For the experiment on robustness presented in Section V-D, we are unable to conduct this experiment on the tree decoder because the modification of the tokens in question texts alters the relation between entities, which further affects some labels essential for the decoding named number group. However, these labels were pre-generated and loaded in the published code of Graph2Tree, which made it impossible for us to reproduce.

APPENDIX C
DETAILED RESULT ANALYSES ON GENERALIZATION EXPERIMENT

An abnormal phenomenon observed in the experiment result in Section V-C is that, in the *number count* split, the accuracy achieved on *number*-0.3 and *number*-0.4, where 30% or 40% of samples are utilized as test data, is even higher than *number*-0.2, where 20% of samples are utilized as test data. In the *formula length* split, these variations are not even monotonic. However, this phenomenon is interpretable. We analyzed the sliding window average accuracy on test samples in these test sets and showed the results in Figure 2 and 3.

In these two figures, the solid gray lines show the count of numbers and the length of formulas of test samples, which serve as evidence for our data sorting and set split. In both figures, the accuracy drops significantly when the count of

TABLE 8. Examples of the poor formula annotations in MathQA and solutions in GSM8K dataset. Some major issues regarding the solutions provided in GSM8K dataset include: a. bad equation formulation (example 3), b. missing calculation step (example 4), c. calculation mistake (example 5). These issues may not fatally affect the learning of the Chain-of-Thought method, yet they will make it impossible to generate formulas from these solutions and thus reduce the available training samples for formula-based methods.

sample	MathQA #1753
question	How many multiples of 4 are there between 8 and 160? **(Answer: 37)**
formula	raw: $[(-, N_2, N_1), (\div, M_0, N_0), (+, M_1, 1)]$ flatten: $((160 - 8) \div 4) + 1$ **oracle:** $((160 - 8) \div 4) - 1$
sample	MathQA #4000
question	The average of first 25 natural numbers is? **(Answer: 13)**
formula	raw: $[(+, N_0, 1)]$ flatten: $25 + 1$ **oracle:** $(25 + 1) \div 2$
sample	GSM8K #7420
question	Tapanga and Corey have 66 candies together. However, Tapanga has 8 more candies than Corey. How many candies does Corey have? **(Answer: 29)**
solution	Let x = the total number of candies Corey has. x + 8 = the total number of candies Tapanga has. The equation for the total number of candies is x + (x + 8) = 66 Combining like terms, we get 2x + 8 = 66 Subtracting 8 from both sides, we get 2x = 58 Dividing both sides by 2, we get x = << 29=29 >> 29, so Corey has 29 candies. (bad equation formulation)
sample	GSM8K #7464
question	An office has 90 workers. 2/3rds of them are men and the rest are women. The company hires 10 new employees and 100% of them are women. What is the total percentage of women in the company now? **(Answer: 40)**
solution	1/3rd of the workers before the new hires were women because 1 - (2/3) = 1/3 There were 30 women working for the company before the new hires There are now 40 women working for the company because 30 + 10 = << 30+10=40 >> 40 There are 100 people working for the company because 90 + 10 = << 90+10=100 >> 100 The proportion of women working in the company is .4 because 40 / 100 = << 40/100=.4 >> .4 The percentage of women working in the company is now 40 because .4 x 100 = << .4*100=40 >> 40 (missing calculation step: how the number 30 is obtained)
sample	GSM8K #7446
question	Mark does a gig every other day for 2 weeks. For each gig, he plays 3 songs. 2 of the songs are 5 minutes long and the last song is twice that long. How many minutes did he play? **(Answer: 140)**
solution	He played 7*2 = << 7*2=14 >> 14 gigs The long song was 5*2= << 5*2=10 >> 10 minutes So all the gigs were 5+5+10= << 5+5+10=20 >> 20 minutes So he played 14*20=140 minutes (calculation mistake, because of wrong reasoning process)

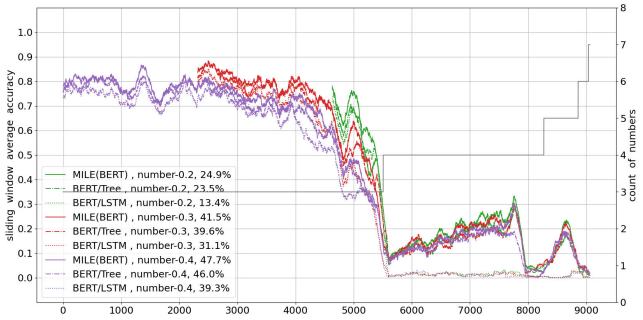


FIGURE 2. The sliding window average accuracy on *number-p* with window width 200.

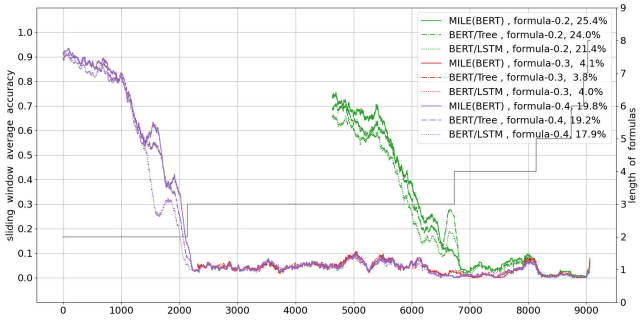


FIGURE 3. The sliding window average accuracy on *formula-p* with window width 200.

numbers or the length of formulas grows. This illustrates the difficulty in generalizing to mathematical problems that have more numbers involved or require longer computational flows to solve. However, compared with *number*-0.2, the 10% or 20% additional test data in *number*-0.3 and *number*-0.4 is of lower difficulty and can be solved with higher accuracy. This is because the numbers in these samples are fewer, and there are also problems with three numbers in

TABLE 9. Examples of the formulas predicted by different inference models on three problems. On these problems, MILE predicted valid formulas that lead to correct answers, whereas the other inference models failed. Note that the questions in Math23K dataset were written in Chinese. We translated them into english only for this demonstration.

test set	*number*-0.2
question	Lee deposits 400 dollars in the bank for 4 years. The annual interest rate is 2.80%. How much after-tax interest can Lee get when it is due? (Deposit interest tax is 20%) **(Answer: 38.54)**
BERT / LSTM	raw: $[\times, \times, N_0, N_1, -, 1, N_2]$ flatten: $(400 \times 4) \times (1 - 2.80\%)$ **(wrong)**
BERT / Tree	raw: $[\times, \times, N_0, N_2, N_1]$ flatten: $(400 \times 2.80\%) \times 4$ **(wrong)**
MILE (BERT)	raw: $[(\times, N_0, N_2), (\times, M_0, N_1), (-, 1, N_3), (\times, M_1, M_2)]$ flatten: $((400 \times 2.80\%) \times 4) \times (1 - 20\%)$
test set	*number*-0.4
question	The distance from Lily's house to the shopping mall is 2400 meters. She ran at a speed of 150 meters per minute for 5 minutes, and then walked at a speed of 60 meters per minute for 16 minutes. How many meters does she still need to walk to reach the shopping mall? **(Answer: 690)**
BERT / LSTM	raw: $[+, N_0, \times, N_1, N_2]$ flatten: $2400 + (150 \times 5)$ **(wrong)**
BERT / Tree	raw: $[-, N_0, \times, N_1, N_4]$ flatten: $2400 - (150 \times 16)$ **(wrong)**
MILE (BERT)	raw: $[(\times, N_1, N_2), (-, N_0, M_0), (\times, N_3, N_4), (-, M_1, M_3)]$ flatten: $2400 - (150 \times 5) - (60 \times 16)$
test set	*formula*-0.2
question	Students in a school go to camp. There are 28 students in the lower grades. There are 17 times as many as lower grades and 16 more students in the higher grades. If every 13 students share a tent, how many tents do the lower grades students and higher grades students need in total? **(Answer: 40)**
BERT / LSTM	raw: $[+, +, N_0, \div, N_0, N_1, N_2]$ flatten: $(28 + (28 \div 17)) + 16$ **(wrong)**
BERT / Tree	raw: $[\div, +, \times, N_0, N_1, N_2, N_3]$ flatten: $((28 \times 17) + 16) \div 13$ **(wrong)**
MILE (BERT)	raw: $[(\times, N_0, N_1), (+, N_2, M_0), (+, N_0, M_1), (\div, M_2, N_2)]$ flatten: $(28 + (16 + (28 \times 17))) \div 13$

the training set, which renders the generalization easier. As a result, including these additional samples into the test set dilutes the total difficulty of testing and makes the final accuracy higher. Nevertheless, if we focus on the overlapping part of test samples in *number*-0.2, *number*-0.3 and *number*-0.4 on the right side, the accuracy achieved on *number*-0.2 is still higher than *number*-0.3 and *number*-0.4, which is a reasonable outcome. The result on the *formula length* split can be explained similarly, whereas the situation is even more radical. The train–test set split on *formula*-0.3 is nearly right on the boundary between problems with formula lengths less than and not less than three. This makes the generalization extremely hard and leads to very low accuracy. In contrast, a portion of the problems with formula length three is split to the training set in *formula*-0.2, and this makes the generalization much easier and the accuracy higher.

A case study of the formulas predicted in this experiment is provided in Table 9.

REFERENCES

[1] D. Saxton, E. Grefenstette, F. Hill, and P. Kohli, "Analysing mathematical reasoning abilities of neural models," in *Proc. Int. Conf. Learn. Represent.*, 2019. [Online]. Available: https://openreview.net/forum?id=H1gR5iR5FX

[2] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," 2021, *arXiv:2110.14168*.

[3] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, no. 240, pp. 1–113, 2023.

[4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain of thought prompting elicits reasoning in large language models,' in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.

[5] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. Tat Lee, Y. Li, S. Lundberg, H. Nori, H. Palangi, M. Tulio Ribeiro, and Y. Zhang, "Sparks of artificial general intelligence: Early experiments with GPT-4," 2023, *arXiv:2303.12712*.

[6] Y. Wang, X. Liu, and S. Shi, "Deep neural solver for math word problems," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2017, pp. 845–854.

[7] L. Wang, "Template-based math word problem solvers with recursive neural networks," in *Proc. 33rd AAAI Conf. Artif. Intell.*, 2019, pp. 7144–7151.

[8] Q. Liu, W. Guan, S. Li, and D. Kawahara, "Tree-structured decoding for solving math word problems," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 2370–2379.

[9] Z. Xie and S. Sun, "A goal-driven tree-structured neural model for math word problems," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 5299–5305.

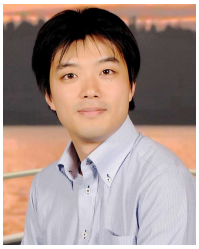
[10] J. Zhang, L. Wang, R. K.-W. Lee, Y. Bin, Y. Wang, J. Shao, and E.-P. Lim, "Graph-to-Tree learning for solving math word problems," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 3928–3937.

[11] A. Amini, S. Gabriel, S. Lin, R. Koncel-Kedziorski, Y. Choi, and H. Hajishirzi, "MathQA: Towards interpretable math word problem solving with operation-based formalisms," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, vol. 1, 2019, pp. 2357–2367.

[12] W. Ling, D. Yogatama, C. Dyer, and P. Blunsom, "Program induction by rationale generation: Learning to solve and explain algebraic word problems," in *Proc. 55th Annu. Meeting Assoc. Comput. Linguistics (Long Papers)*, vol. 1, 2017, pp. 159–167.

[13] K. Chen, Q. Huang, H. Palangi, P. Smolensky, K. Forbus, and J. Gao, "Mapping natural-language problems to formal-language solutions using structured neural representations," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 1566–1575.
[14] Y. Wu and H. Nakayama, "Weakly supervised formula learner for solving mathematical problems," in *Proc. 29th Int. Conf. Comput. Linguistics*, 2022, pp. 1743–1752.
[15] J. Shen, Y. Yin, L. Li, L. Shang, X. Jiang, M. Zhang, and Q. Liu, "Generate & rank: A multi-task framework for math word problems," in *Proc. Findings Assoc. Comput. Linguistics (EMNLP)*, 2021, pp. 2269–2279.
[16] Z. Liang, J. Zhang, L. Wang, W. Qin, Y. Lan, J. Shao, and X. Zhang, "MWP-BERT: Numeracy-augmented pre-training for math word problem solving," in *Proc. Findings Assoc. Comput. Linguistics (NAACL)*, 2022, pp. 997–1009.
[17] Z. Jie, J. Li, and W. Lu, "Learning to reason deductively: Math word problem solving as complex relation extraction," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (Long Papers)*, vol. 1, 2022, pp. 5944–5955.
[18] V. Shwartz, P. West, R. Le Bras, C. Bhagavatula, and Y. Choi, "Unsupervised commonsense question answering with self-talk," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 4615–4629.
[19] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," in *Proc. Extended Abstr. CHI Conf. Hum. Factors Comput. Syst.*, May 2021, pp. 1–7.
[20] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 22199–22213.
[21] Stephen Wolfram. *ChatGPT Gets Its 'Wolfram Superpowers'*. Accessed: Mar. 25, 2024. [Online]. Available: https://platform.openai.com/docs/guides/gpt
[22] A. Graves, G. Wayne, and I. Danihelka, "Neural Turing machines," 2014, *arXiv:1410.5401*.
[23] A. Graves, G. Wayne, M. Reynolds, T. Harley, I. Danihelka, A. Grabska-Barwińska, S. G. Colmenarejo, E. Grefenstette, T. Ramalho, J. Agapiou, A. P. Badia, K. M. Hermann, Y. Zwols, G. Ostrovski, A. Cain, H. King, C. Summerfield, P. Blunsom, K. Kavukcuoglu, and D. Hassabis, "Hybrid computing using a neural network with dynamic external memory," *Nature*, vol. 538, no. 7626, pp. 471–476, Oct. 2016.
[24] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
[25] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, vol. 1, Jun. 2019, pp. 4171–4186.
[26] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
[27] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *Proc. NIPS Workshop Deep Learn.*, 2014.
[28] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
[29] R. Koncel-Kedziorski, S. Roy, A. Amini, N. Kushman, and H. Hajishirzi, "MAWPS: A math word problem repository," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2016, pp. 1152–1156.
[30] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Jan. 1997.
[31] Hugging Face. *Bert-Base-Chinese*. Accessed: Dec. 27, 2023. [Online]. Available: https://huggingface.co/bert-base-chinese
[32] Y. Cui, W. Che, T. Liu, B. Qin, S. Wang, and G. Hu, "Revisiting pre-trained models for Chinese natural language processing," in *Proc. Findings Assoc. Comput. Linguistics (EMNLP)*, 2020, pp. 657–668.
[33] Hugging Face. *Chinese-Roberta-WWM-Ext*. Accessed: Dec. 27, 2023. [Online]. Available: https://huggingface.co/hfl/chinese-roberta-wwm-ext
[34] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent.*, 2015, pp. 1–15.

YUXUAN WU received the master's degree in information science from The University of Tokyo, Japan, in 2020, where he is currently pursuing the Ph.D. degree with the Nakayama Laboratory, Graduate School of Information Science and Technology. His research interests include neuro-symbolic methods in machine learning, deep learning, and reinforcement learning.



HIDEKI NAKAYAMA (Member, IEEE) received the master's and Ph.D. degrees in information science The University of Tokyo, Japan, in 2008 and 2011, respectively. He is a Professor at the Department of Creative Informatics, School of Information Science and Technology, The University of Tokyo. He joined the current department as an Assistant Professor in 2012, where he has been a Full Professor since 2024. He is also a Faculty Member with the Institute of AI and Beyond of the University of Tokyo, and the International Research Center for Neurointelligence (IRCN), respectively. His research interests include visual recognition and generation, natural language processing, and multimodal learning.